System Architecture

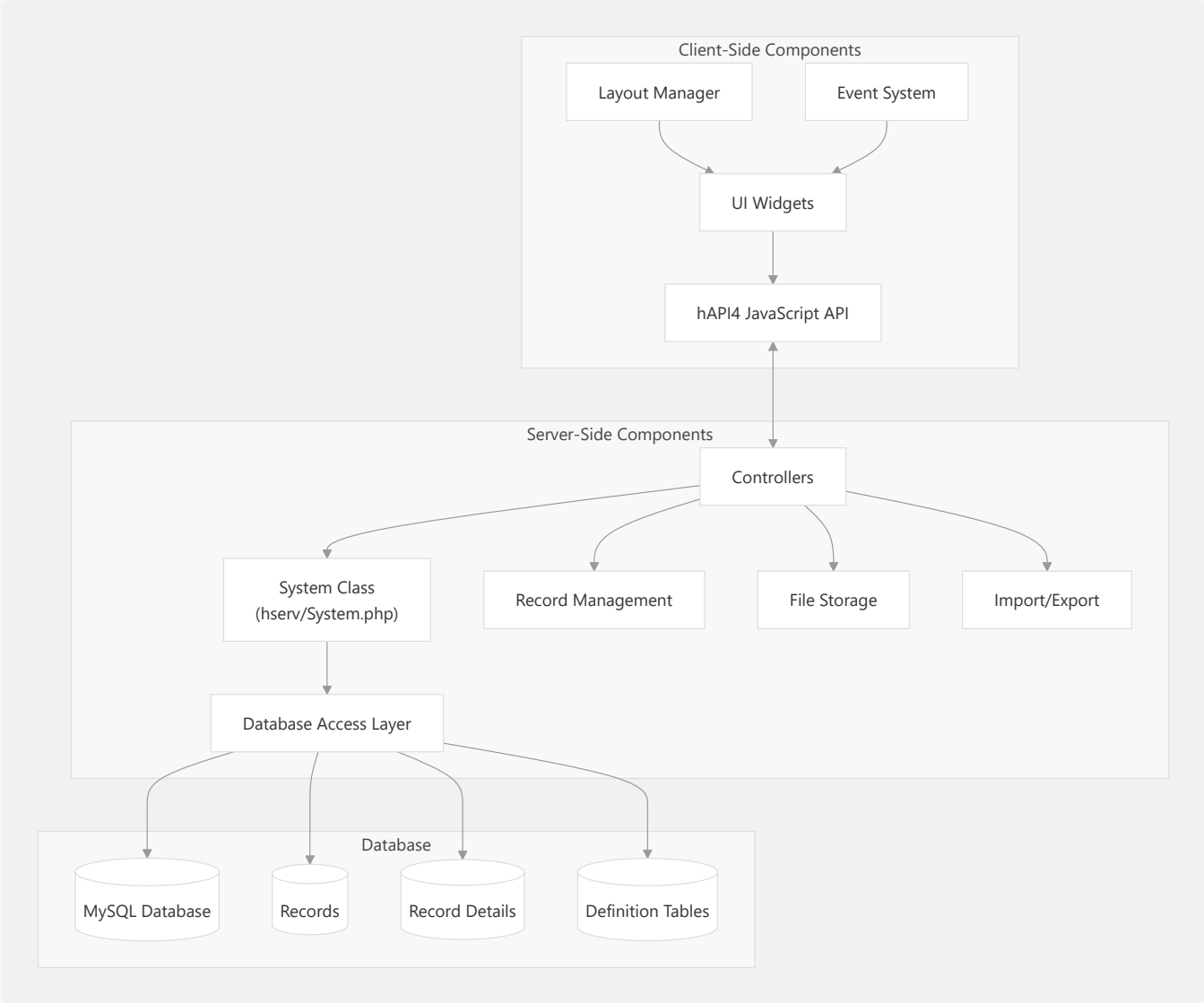
Relevant source files

- admin/utilities/downloadInteractionLog.php
- hclient/core/layout.js
- hclient/framecontent/initPage.php
- hserv/System.php
- hserv/consts.php
- hserv/controller/usr_info.php
- hserv/dbaccess/utils_db.php
- hserv/records/edit/recordModify.php
- hserv/records/edit/recordTitleMask.php
- hserv/records/edit/recordsBatch.php
- hserv/records/import/importAction.php
- hserv/records/import/importParser.php
- hserv/records/search/recordSearch.php
- hserv/structure/dbsUsersGroups.php
- hserv/structure/search/dbsData.php
- hserv/utilities/utils_db_load_script.php
- index.php
- layout_default.js
- redirects/resolver.php
- viewers/record/viewRecord.php

This document provides an overview of the Heurist system architecture, explaining the core components and their interactions. It covers both server-side and client-side components, the initialization process, and data flow between components.

Core Architecture Overview

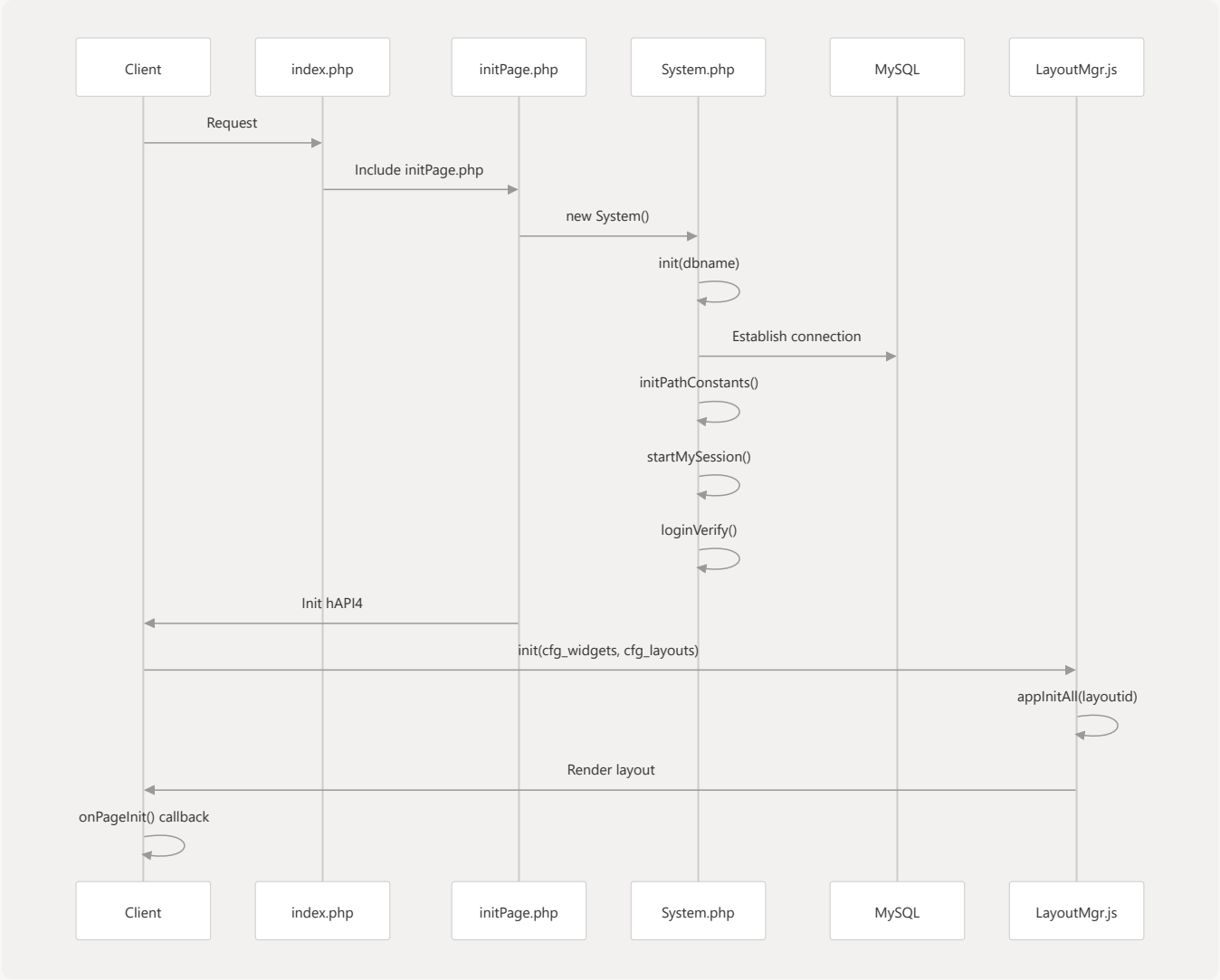
Heurist is built on a client-server architecture with a PHP backend, MySQL database, and JavaScript frontend. The system is organized into several key subsystems that work together to provide a comprehensive knowledge management platform.



Sources: `index.php 1-584` `hserv/System.php 1-897` `hclient/framecontent/initPage.php 1-476`

System Initialization Process

When a user accesses Heurist, the system goes through a specific initialization sequence to establish database connections, authenticate users, and set up the UI framework.



Sources: `index.php` `364-420` `hclient/framecontent/initPage.php` `44-175` `hserv/System.php` `96-144`

Core System Components

System Class

The `System` class is the central component of the Heurist backend. It handles database connections, user authentication, system settings, and file system operations.

System
-mysqli -dbnameFull -dbname -errors[] -isInited -currentUser -settings
+__construct(full_check) +init(db, dbrequired, init_session_and_constants) +dbclose() +defineConstants(reset) +initPathConstants(dbname) +getMysqli() +loginVerify() +hasAccess() +isAdmin() +isMember(groups) +isDbOwner() +getUserId() +getCurrentUser() +errorExit(message, error_code)

Sources: `hserv/System.php` `46-897`

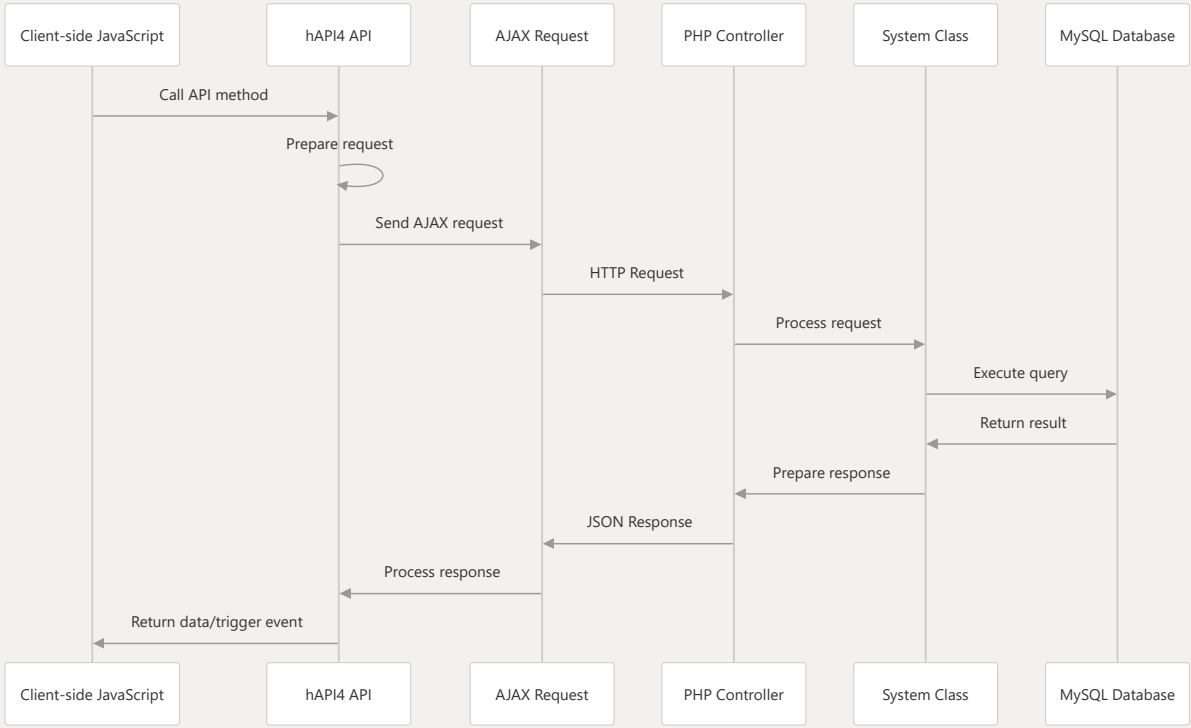
Database Access Layer

The database layer provides utilities for connecting to MySQL and executing queries. It handles connection management, query execution, and result processing.

Sources: `hserv/dbaccess/utils_db.php` `84-169` `hserv/dbaccess/utils_db.php` `371-516`

Client-Server Communication

The client communicates with the server through AJAX requests managed by the hAPI4 JavaScript API. This API provides methods for searching records, managing user sessions, and handling system events.



Sources: `hclient/framecontent/initPage.php` `290-344` `hserv/controller/usr_info.php` `47-94`

Record Management

Records are a core concept in Heurist. The record management system handles creating, updating, retrieving, and deleting records, as well as managing their structure and relationships.

Sources: `hserv/records/edit/recordModify.php` `171-290`
`hserv/records/edit/recordModify.php` `330-573` `hserv/records/search/recordSearch.php` `92-133`

User Interface Architecture

Heurist's UI is built on a flexible layout system that arranges widgets into panes. The layout is configured in JavaScript and managed by the LayoutManager.

Sources: `layout_default.js` `40-297` `hclient/core/layout.js` `29-377`

Configuration System

Heurist uses a hierarchical configuration system that includes server-side PHP settings and client-side JavaScript configurations.

Sources: `hserv/consts.php 1-188` `layout_default.js 40-101` `layout_default.js 99-297`

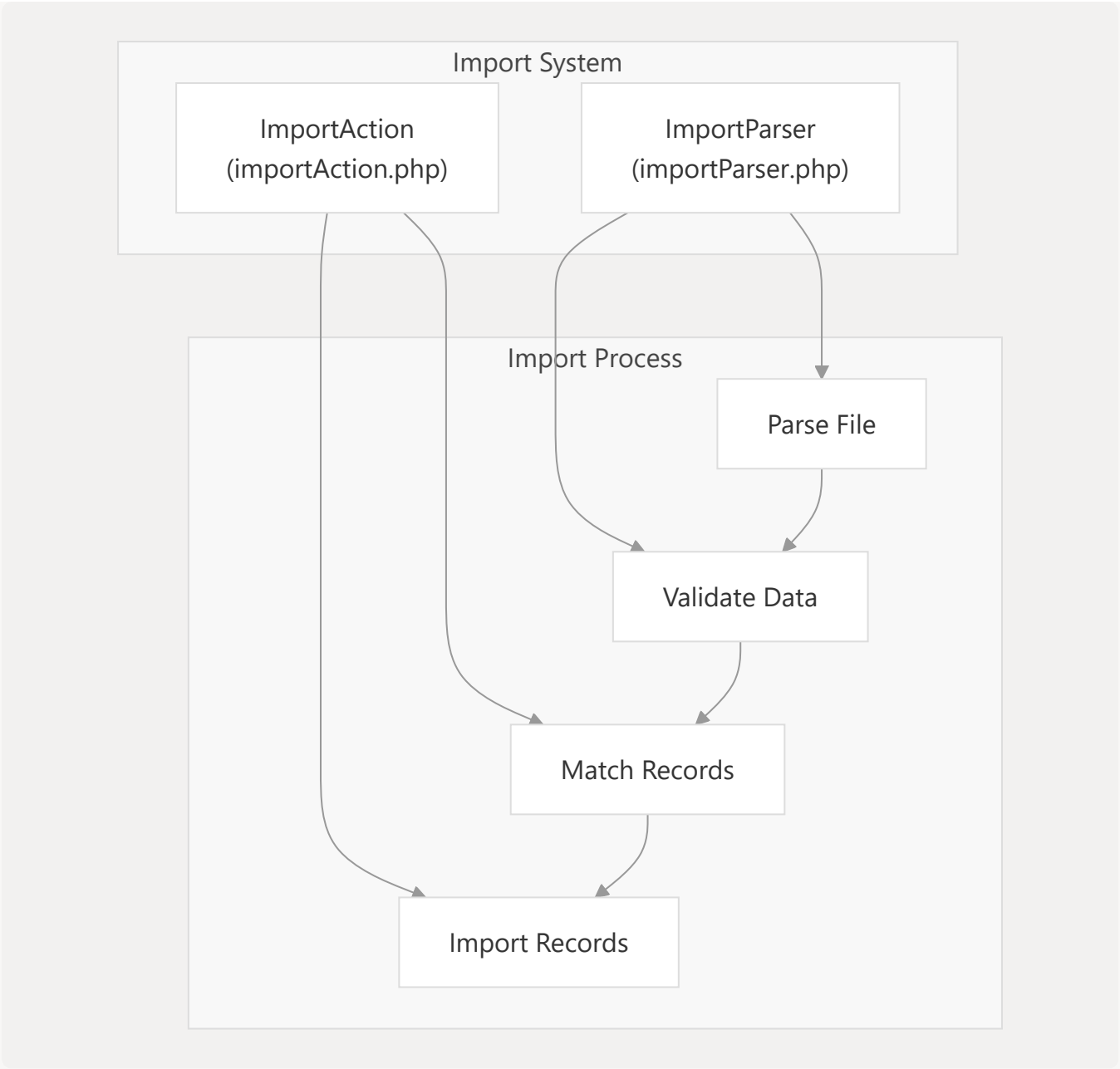
URL Resolution System

Heurist implements a URL resolution system that maps semantic URLs to internal application routes, making it possible to link directly to records and other resources.

Sources: `redirects/resolver.php 56-175` `redirects/resolver.php 177-259`

Import/Export System

Heurist provides comprehensive import and export capabilities for data interchange.



Sources: `hserv/records/import/importParser.php` `35-86`
`hserv/records/import/importAction.php` `47-106`

Batch Operations

Heurist supports batch operations on records for efficient management of large datasets.

Sources: `hserv/records/edit/recordsBatch.php` `61-119`

Data Model

The core data model in Heurist revolves around records, their details, and the definition of record types and field types.